

```

/**
 * Datacake Decoder for Milesight WTS506 Weather Station
 * Hardened against rainfall spikes and negative deltas
 */
function Decoder(bytes, port) {
  var decoded = {};
  var i = 0;

  while (i < bytes.length) {

    // Guard: need at least 2 bytes for channel + type
    if (i + 1 >= bytes.length) break;

    var channel = bytes[i++];
    var type = bytes[i++];

    // BATTERY
    if (channel === 0x01 && type === 0x75) {
      if (i + 1 > bytes.length) break;
      decoded.BATTERY = bytes[i++];
    }
    // TEMPERATURE (°C) — signed Int16LE / 10
    else if (channel === 0x03 && type === 0x67) {
      if (i + 2 > bytes.length) break;
      var v = readInt16LE(bytes, i);
      decoded.TEMPERATURE = v / 10;
      i += 2;
    }
    // HUMIDITY (%)
    else if (channel === 0x04 && type === 0x68) {
      if (i + 1 > bytes.length) break;
      decoded.HUMIDITY = bytes[i++] / 2;
    }
    // WIND DIRECTION (°)
    else if (channel === 0x05 && type === 0x84) {
      if (i + 2 > bytes.length) break;
      var v = readUInt16LE(bytes, i);
      decoded.WIND_DIRECTION = v / 10;
      decoded.WIND_DIRECTION_TEXT = getWindDirection(decoded.WIND_DIRECTION);
      i += 2;
    }
    // PRESSURE (hPa)
    else if (channel === 0x06 && type === 0x73) {
      if (i + 2 > bytes.length) break;
      decoded.PRESSURE = readUInt16LE(bytes, i) / 10;
      i += 2;
    }
    // WIND SPEED (m/s)
    else if (channel === 0x07 && type === 0x92) {
      if (i + 2 > bytes.length) break;
      decoded.WIND_SPEED = readUInt16LE(bytes, i) / 10;
      i += 2;
    }
    // RAINFALL (mm) — 4-byte counter + 1-byte rain_counter
    else if (channel === 0x08 && type === 0xEC) {
      if (i + 5 > bytes.length) break;

      // Decode raw cumulative rainfall from device (UInt32LE / 100)
      var raw_now = readUInt32LE(bytes, i) / 100;
      var rain_counter = bytes[i + 4];
    }
  }
}

```

```

// Read previous values persisted in Datacake DB
var raw_prev = (typeof measurements !== "undefined" &&
    measurements.RAIN_TODAY_RAW &&
    measurements.RAIN_TODAY_RAW.value !== null)
    ? measurements.RAIN_TODAY_RAW.value
    : null;

var total_prev = (typeof measurements !== "undefined" &&
    measurements.RAIN_TOTAL &&
    measurements.RAIN_TOTAL.value !== null)
    ? measurements.RAIN_TOTAL.value
    : 0;

// Compute increment — handle first message, reset, and rollover
var inc = 0;
if (raw_prev !== null) {
    inc = raw_now - raw_prev;
    if (inc < 0) inc = raw_now; // counter reset / new day
}
if (inc < 0) inc = 0;

// Reject obvious spikes (> 30 mm per uplink interval)
if (inc > 30) {
    decoded.RAIN_REJECTED_SPIKE = true; // ← flag kept as boolean for Datacake field
    inc = 0;
} else {
    decoded.RAIN_REJECTED_SPIKE = false; // ← always emit so field stays fresh in DB
}

decoded.RAIN_TODAY_RAW = parseFloat(raw_now.toFixed(2));
decoded.RAIN_INCREMENT = parseFloat(inc.toFixed(2));
decoded.RAIN_TOTAL = parseFloat((total_prev + inc).toFixed(2));
decoded.RAIN_COUNTER = rain_counter; // ← uppercased to match Datacake
convention

    i += 5;
}
// Unknown channel — skip byte and continue
else {
    i++;
}
}

// LoRa radio metadata
try {
    var gw = (normalizedPayload.gateways && normalizedPayload.gateways[0]) || {};
    decoded.LORA_RSSI = gw.rssi || 0;
    decoded.LORA_SNR = gw.snr || 0;
    decoded.LORA_DATARATE = normalizedPayload.data_rate || "not retrievable";
} catch (e) {
    decoded.LORA_RSSI = 0;
    decoded.LORA_SNR = 0;
    decoded.LORA_DATARATE = "not retrievable";
}

// Datacake expects [{ field: "FIELD_NAME", value: ... }]
return Object.keys(decoded).map(function(key) {
    return { field: key, value: decoded[key] }; // keys already uppercase
});

```

```

}

/* *****
* Byte helpers — index-based (no slice GC)
***** */
function readUInt16LE(bytes, i) {
    return (bytes[i] | (bytes[i + 1] << 8)) & 0xFFFF;
}
function readInt16LE(bytes, i) {
    var v = readUInt16LE(bytes, i);
    return v > 0x7FFF ? v - 0x10000 : v;
}
function readUInt32LE(bytes, i) {
    return (((bytes[i + 3] << 24) |
        (bytes[i + 2] << 16) |
        (bytes[i + 1] << 8) |
        bytes[i]) >>> 0); // >>> 0 forces unsigned
}

/* *****
* Wind direction compass label
***** */
function getWindDirection(deg) {
    var d = ['N', 'NNE', 'NE', 'ENE', 'E', 'ESE', 'SE', 'SSE',
        'S', 'SSW', 'SW', 'WSW', 'W', 'WNW', 'NW', 'NNW'];
    return d[Math.round(deg / 22.5) % 16];
}

```